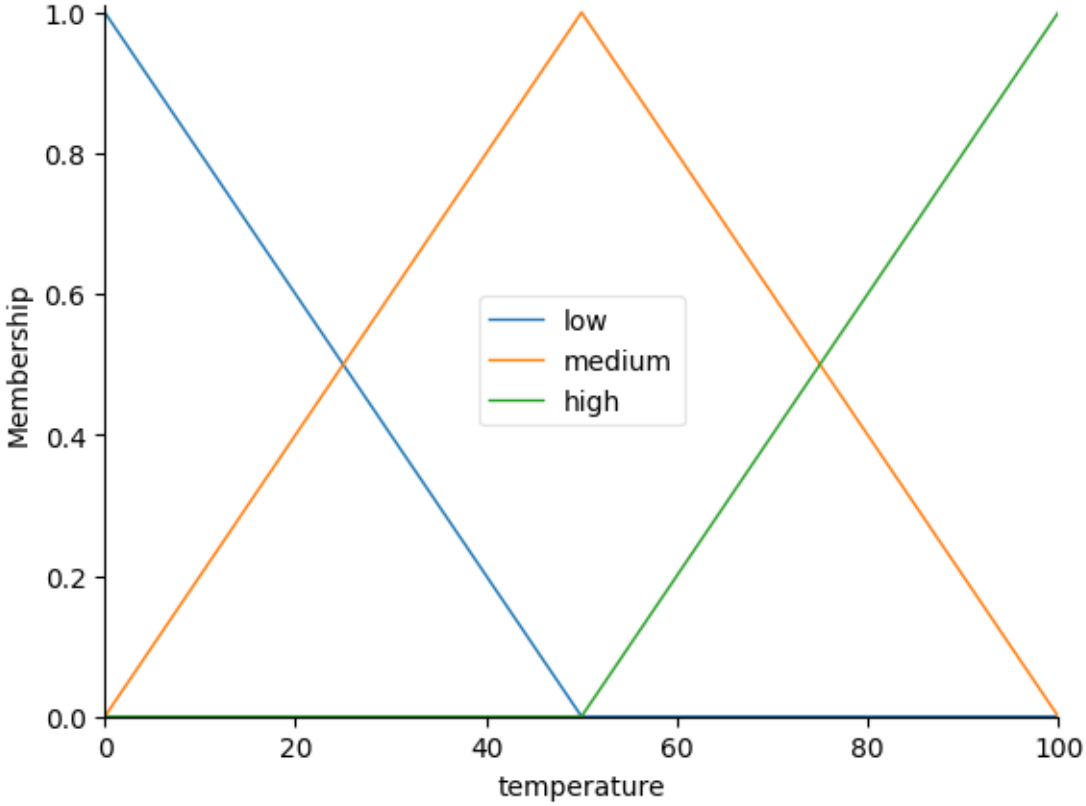
SOFT COMPUTING

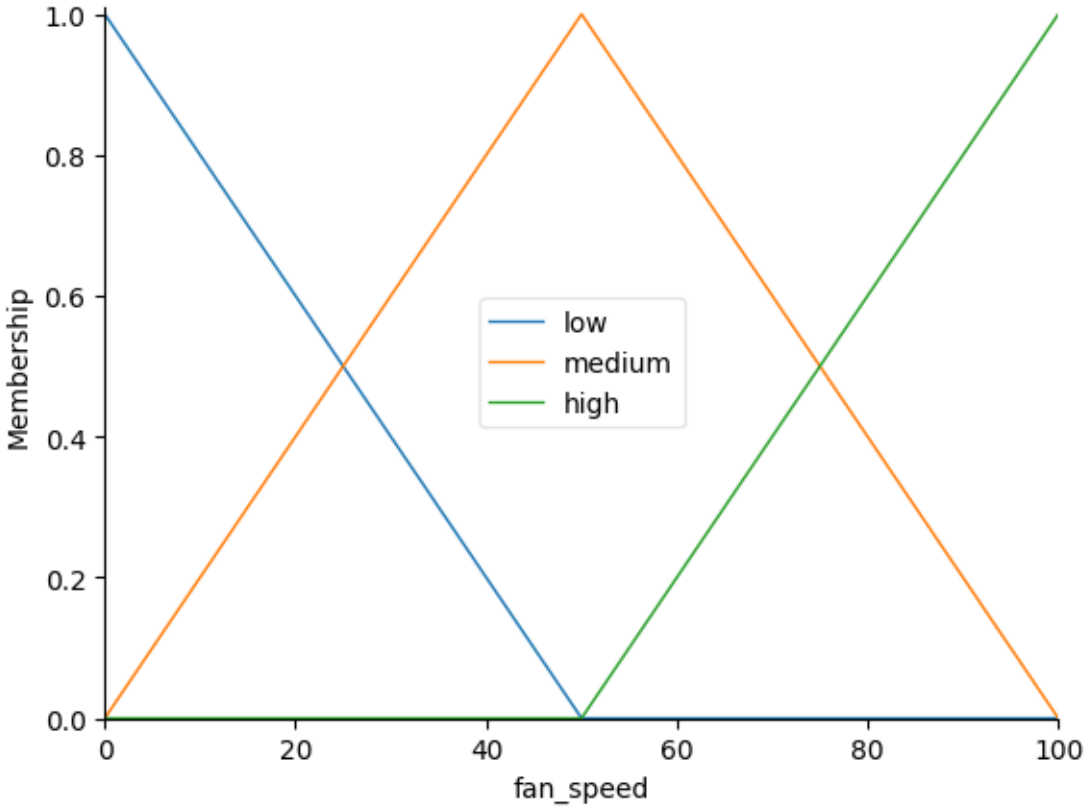
Ex : 01 Implementation of fuzzy control inference system

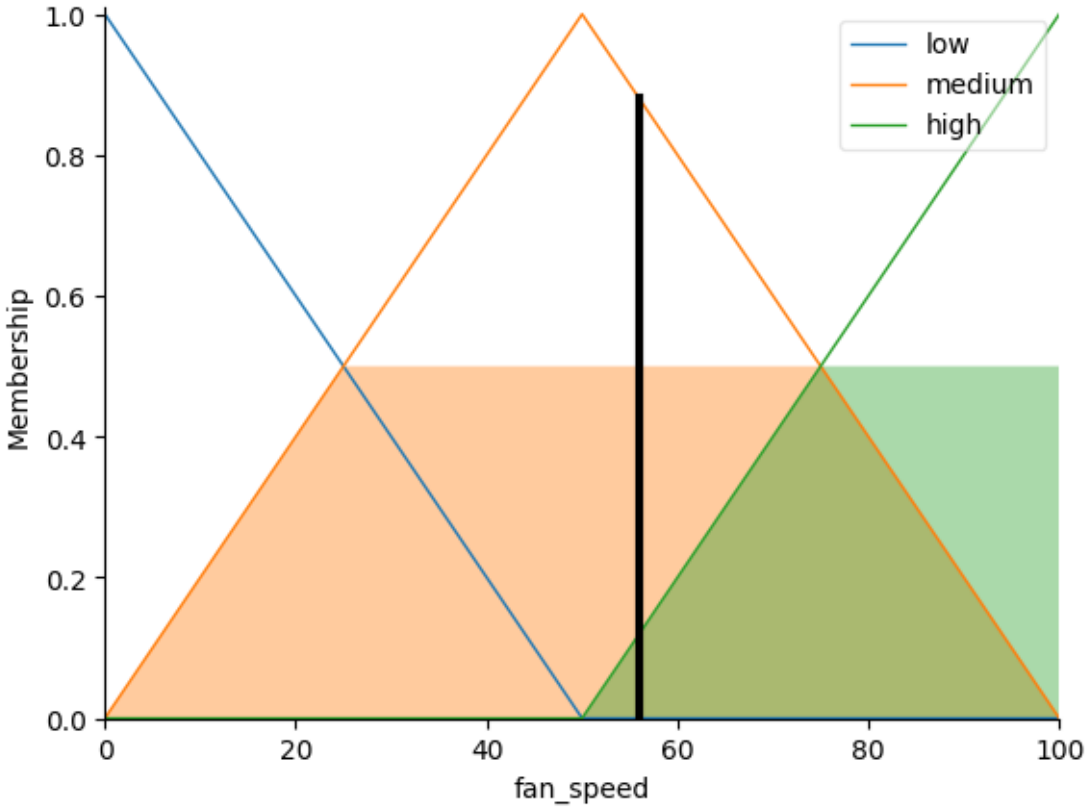
```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

temperature=ctrl.Antecedent(np.arange(0,101,1),'temperature')
fan_speed=ctrl.Consequent(np.arange(0,101,1),'fan_speed')

temperature['low']=fuzz.trimf(temperature.universe,[0,0,50])
temperature['medium']=fuzz.trimf(temperature.universe,[0,50,100])
temperature['high']=fuzz.trimf(temperature.universe,[50,100,100])

fan_speed['low']=fuzz.trimf(fan_speed.universe,[0,0,50])
fan_speed['medium']=fuzz.trimf(fan_speed.universe,[0,50,100])
fan_speed['high']=fuzz.trimf(fan_speed.universe,[50,100,100])

rule1=ctrl.Rule(temperature['low'],fan_speed['low'])
rule2=ctrl.Rule(temperature['medium'],fan_speed['medium'])
rule3=ctrl.Rule(temperature['high'],fan_speed['high'])

fan_ctrl=ctrl.ControlSystem([rule1,rule2,rule3])
fan_speed_ctrl=ctrl.ControlSystemSimulation(fan_ctrl)

temperatur_value=75
fan_speed_ctrl.input['temperature']=temperatur_value
fan_speed_ctrl.compute()

print("Fan speed:",fan_speed_ctrl.output['fan_speed'])
temperature.view()
fan_speed.view()
fan_speed.view(sim=fan_speed_ctrl)
```

OUTPUT:

Fan speed: 55.95238095238095







EX : 02 Programming exercise on classification with a discrete perceptron

```
import numpy as np

X = np.array([[2,3],[3,2],[1,1],[5,7],[6,8],[7,6]])
y = np.array([0,0,0,1,1,1])

w = np.zeros(2)
b = 0

for _ in range(100):
    for xi, yi in zip(X, y):
        output = np.dot(w, xi) + b
        pred = 1 if output > 0 else 0
        error = yi - pred
        w += 0.1 * error * xi
        b += 0.1 * error

test_data = np.array([[4,5],[2,2]])
```

```
for data in test_data:
    output = np.dot(w, data) + b
    pred = 1 if output > 0 else 0
    if pred == 0:
        print(f'Data {data} belongs to class 0")
    else:
        print(f'Data {data} belongs to class 1")
```

OUTPUT:

Data [4 5] belongs to class 1

Data [2 2] belongs to class 0

EX : 03 Implementation of XOR with backpropagation algorithm

```
import numpy as np
def sigmoid(x):
    return 1/(1+np.exp(-x))

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

w1 = np.random.rand(2,2)
w2 = np.random.rand(2,1)

for _ in range(10000):
    h = sigmoid(np.dot(X, w1))
    o = sigmoid(np.dot(h, w2))

    d2 = (y - o) * o * (1 - o)
    d1 = d2.dot(w2.T) * h * (1 - h)

    w2 += h.T.dot(d2) * 0.1
    w1 += X.T.dot(d1) * 0.1

for i in X:
```

```
    h = sigmoid(np.dot(i, w1))
    o = sigmoid(np.dot(h, w2))
    print(f"Input: {i} Predicted Output: {o[0]}")
```

OUTPUT:

Input: [0 0] Predicted Output: 0.20828212144459304

Input: [0 1] Predicted Output: 0.7320811277505185

Input: [1 0] Predicted Output: 0.7320564702881848

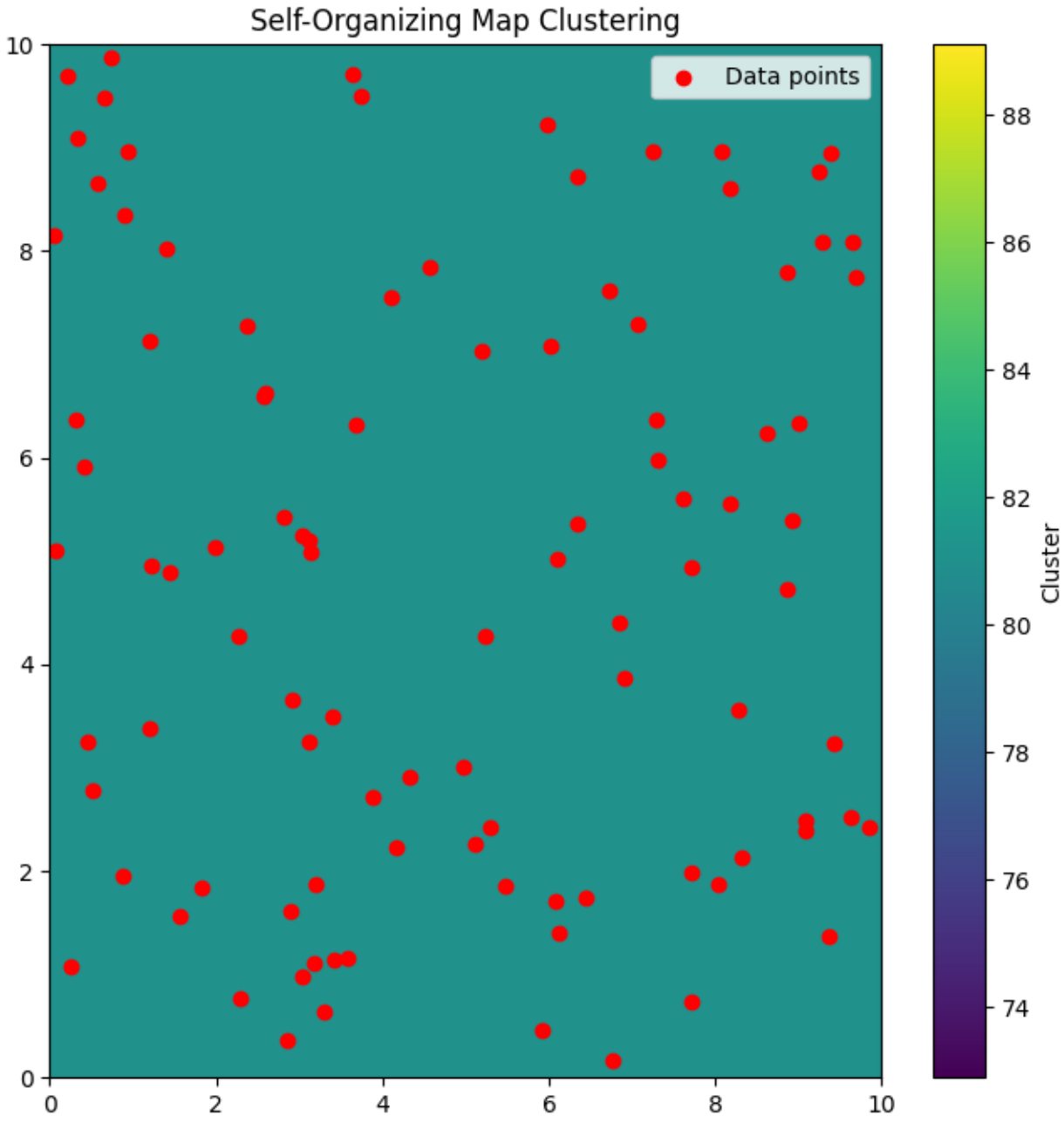
Input: [1 1] Predicted Output: 0.34865730145097934

EX : 04 Implementation of self-organizing maps for a specific application.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
data = np.random.rand(100, 2)

grid = (10, 10)
w = np.random.rand(grid[0], grid[1], 2)

for _ in range(500):
    for x in data:
        d = np.linalg.norm(w - x, axis=2)
        bmu = np.unravel_index(np.argmin(d), d.shape)

        for i in range(grid[0]):
            for j in range(grid[1]):
                dist = np.linalg.norm([i - bmu[0], j - bmu[1]])
                influence = np.exp(-dist**2 / 10)
                w[i, j] += 0.2 * influence * (x - w[i, j])

cluster = np.zeros(grid)

for i in range(grid[0]):
    for j in range(grid[1]):
```

```
        d = np.linalg.norm(data - w[i, j], axis=1)
        cluster[i, j] = np.argmin(d)

plt.pcolormesh(cluster, cmap='viridis')
plt.colorbar()
plt.scatter(data[:,0]*10, data[:,1]*10, c='red')
plt.title("Self-Organizing Map Clustering")
plt.show()
```

OUTPUT:



EX : 05 Programming exercises on maximizing a function using Genetic algorithm.

```
import random

def f(x):
    return -x**2 + 6*x + 9

# Initial population
pop = [random.uniform(-10, 10) for _ in range(50)]

for gen in range(50):
    new_pop = []

    for _ in range(50):
        p1 = random.choice(pop)
        p2 = random.choice(pop)

        # Crossover
        c = (p1 + p2) / 2

        # Mutation
        if random.random() < 0.01:
            c += random.uniform(-1, 1)

        new_pop.append(c)

    pop = new_pop

    best = max(pop, key=f)
    print(f"Generation {gen+1}: Best individual - {best}, Fitness - {f(best)}")

best = max(pop, key=f)
print(f"Best solution found: {best}, Fitness: {f(best)}")
```

OUTPUT:

Generation 1: Best individual-2.5368453802444915,Fitness-17.78548779819913

Generation 2: Best individual-3.3757656312628463,Fitness-17.858800190361634

Generation 3: Best individual-2.9500780404814306,Fitness-17.997507797957823

Generation 4: Best individual-2.9500780404814306,Fitness-17.997507797957823

Generation 5: Best individual-3.044518750106739,Fitness-17.998018080888933

Generation 6: Best individual-3.0821817425498086,Fitness-17.99324616119148

Generation 7: Best individual-2.952049849447671,Fitness-17.997700783062008

Generation 8: Best individual-3.000873047661292,Fitness-17.999999237787783

Generation 9: Best individual-3.000873047661292,Fitness-17.999999237787783

Generation 10: Best individual-2.9697972580376497,Fitness-17.999087794377957

Generation 11: Best individual-2.9697972580376497,Fitness-17.999087794377957

Generation 12: Best individual-3.0205695956892114,Fitness-17.99957689173318

Generation 13: Best individual-3.0205695956892114,Fitness-17.99957689173318

Generation 14: Best individual-3.0205695956892114,Fitness-17.99957689173318

Generation 15: Best individual-2.8568817927833328,Fitness-17.979517178763086

Generation 16: Best individual-2.8568817927833328,Fitness-17.979517178763086

Generation 17: Best individual-2.8568817927833328,Fitness-17.979517178763086

Generation 18: Best individual-2.831995876142457,Fitness-17.971774614366858

Generation 19: Best individual-2.7504688508915924,Fitness-17.937734205624636

Generation 20: Best individual-2.7504688508915924,Fitness-17.937734205624636

Best solution found:2.7504688508915924,Fitness-17.937734205624636

EX : 06 Implementation of two input sine function.

```
import random, math

def f(x, y):
    return math.sin(x) + math.sin(y)

# Initial population
pop = [(random.uniform(-2*math.pi, 2*math.pi),
        random.uniform(-2*math.pi, 2*math.pi)) for _ in range(100)]

for gen in range(20):
    new_pop = []
```

```
    for _ in range(50):
        p1 = random.choice(pop)
        p2 = random.choice(pop)

        # Crossover
        c = (p1[0], p2[1])

        # Mutation
        if random.random() < 0.01:
            c = (c[0] + random.uniform(-0.1, 0.1),
                 c[1] + random.uniform(-0.1, 0.1))

        new_pop.append(c)

    pop = new_pop

    best = max(pop, key=lambda i: f(i[0], i[1]))
    print(f"Generation {gen+1}: Best individual - {best}, Fitness - {f(best[0], best[1])}")

best = max(pop, key=lambda i: f(i[0], i[1]))
print(f"Best solution found: {best}, Fitness: {f(best[0], best[1])}")
```

OUTPUT:

Generation 1: Best individual - (-4.694874824520438, -4.967824214917141), Fitness - 1.967400049220439

Generation 2: Best individual - (-4.368077084979669, 1.2957895599518379), Fitness - 1.9037313045725401

Generation 3: Best individual - (-4.694874824520438, -4.8979985983200915), Fitness - 1.982670561977589

Generation 4: Best individual - (-4.694874824520438, -4.721524899018712), Fitness - 1.9998048988783013

Generation 5: Best individual - (-4.694874824520438, -4.721524899018712), Fitness - 1.9998048988783013

Generation 6: Best individual - (-4.694874824520438, -5.0172987965291185), Fitness - 1.9537206613778317

Generation 7: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 8: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 9: Best individual - (-4.694874824520438, -4.721524899018712), Fitness - 1.9998048988783013

Generation 10: Best individual - (2.687604019686555, 1.8648761206892974), Fitness - 1.395622896224685

Generation 11: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 12: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 13: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 14: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 15: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 16: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 17: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 18: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 19: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Generation 20: Best individual - (-4.694874824520438, 1.8648761206892974), Fitness - 1.9569159088610748

Best solution found: (-4.694874824520438, 1.8648761206892974), Fitness: 1.9569159088610748

EX : 07 Implementation of three input nonlinear function

```
import random
import math


# Fitness function
def fitness(x, y, z):
    return -(x**2+y**2+z**2) + 10*(math.cos(2*math.pi*x)+
                            math.cos(2*math.pi*y)+
                            math.cos(2*math.pi*z))


# 20 initial solutions
population = [(random.uniform(-1,1),
```

```
            random.uniform(-1,1),
            random.uniform(-1,1)) for _ in range(20)]

# 20 generations
for gen in range(20):
    new_pop=[]

    for _ in range(20):
        p1=random.choice(population)
        p2=random.choice(population)

        # Crossover
        child=((p1[0]+p2[0])/2,
               (p1[1]+p2[1])/2,
               (p1[2]+p2[2])/2)

        # Mutation
        if random.random()<0.1:
            child=(child[0]+random.uniform(-0.1,0.1),
                   child[1]+random.uniform(-0.1,0.1),
                   child[2]+random.uniform(-0.1,0.1))

        new_pop.append(child)

    population=new_pop

    best=max(population,key=lambda i: fitness(i[0],i[1],i[2]))

    print(f"Generation {gen+1}: Best individual - {best}, Fitness - {fitness(*best)}")

# Final Best Solution
best=max(population,key=lambda i: fitness(i[0],i[1],i[2]))
print(f"Best solution found: {best}, Fitness: {fitness(*best)}")
```

OUTPUT:

Generation 1: Best individual - (-0.038918299922201416, 0.9455698622405653, 0.213815455498494), Fitness - 20.436058027005927

Generation 2: Best individual - (0.18374279928141518, -0.018307727420266942, -0.012729120116805243), Fitness - 23.911537084294586

Generation 3: Best individual - (0.23355970927735908, 0.05215338184473847, -0.03220640711120748), Fitness - 20.236668403571684

Generation 4: Best individual - (0.15578872640234886, 0.16456455224800393, 0.01799530101788005), Fitness - 20.578193115417488

Generation 5: Best individual - (0.02632425760724485, 0.1663917709455566, -0.08356485453089758), Fitness - 23.49608530082438

Generation 6: Best individual - (0.10580001839585967, 0.130898686917015, -0.07312268340785175), Fitness - 23.604161433799973

Generation 7: Best individual - (0.0837156497071628, 0.12139110138180734, -0.14690324621986817), Fitness - 21.86861325168183

Generation 8: Best individual - (0.03379900840689998, 0.15193475865595252, -0.1722966794711615), Fitness - 20.19109546817838

Generation 9: Best individual - (0.03430378428480651, 0.16868481497414195, -0.1872607573368743), Fitness - 18.4344221693639

Generation 10: Best individual - (0.028175138858728882, 0.17422333084194075, -0.20522687693938435), Fitness - 17.129996345963455

Generation 11: Best individual - (0.05961422189599338, 0.13090637348682063, -0.20650634825168024), Fitness - 18.746070385453777

Generation 12: Best individual - (0.03965475011262101, 0.16243188613448276, -0.21571678881937073), Fitness - 16.982805360700276

Generation 13: Best individual - (0.012629098625507727, 0.09455272156520561, -0.23398268833396363), Fitness - 19.195954878259304

Generation 14: Best individual - (0.02310750707974682, 0.10117650266260012, -0.22255309940275983), Fitness - 19.59698901489344

Generation 15: Best individual - (0.030032981354552263, 0.1474434633776079, -0.2398609447392065), Fitness - 16.38597657057121

Generation 16: Best individual - (0.0727410999912067, 0.17639487658209901, -0.20271436409074994), Fitness - 16.285269942749636

Generation 17: Best individual - (0.00859446859345489, 0.15138474804725258, -0.16394543826355756), Fitness - 20.890130074540046

Generation 18: Best individual - (0.00859446859345489, 0.15138474804725258, -0.16394543826355756), Fitness - 20.890130074540046

Generation 19: Best individual - (0.02729640991306274, 0.16241369273878492, -0.202772947198823), Fitness - 17.938660250779147

Generation 20: Best individual - (0.0570279416805592, 0.15458818912160577, -0.2000056381333149), Fitness - 18.029779908422267

Best solution found: (0.0570279416805592, 0.15458818912160577, -0.2000056381333149), Fitness: 18.029779908422267